

Accelerating NTRU-based Bootstrapping with Block Key Distributions

Jingwei Feng^{1,2}, Baofeng Wu^{1,2}, Dongdai Lin^{1,2}, and Binwu Xiang³

¹ Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, CAS, Beijing, China

{fengjingwei, wubaofeng, ddlin}@iie.ac.cn

² School of Cyber Security, University of Chinese Academy of Sciences, Beijing, China

³ East China Normal University, Shanghai, China

xiangbinwu@163.com

Abstract. NTRU-based bootstrapping is a high-performance variant of FHEW-like bootstrapping schemes. Its main computational bottleneck lies in the blind rotation step, which involves numerous external products. In this work, we propose multiple techniques to reduce the number of these costly operations, including the use of block binary keys, block ternary keys, and the integration of block keys with the key unrolling method. Specifically, our approach reduces the number of external products to n/ℓ for block keys $\mathbf{B}_{k,\ell}$ ($\mathbf{T}_{k,\ell}$), compared to n in FINAL (ASIACRYPT 2021) and $n/2$ in the key unrolling approach (CHES 2024). When combining block keys with key unrolling, the number can further be reduced to $n/2\ell$.

We implemented our algorithms using the CHIFFE library. Under 128-bit security settings, using block binary keys $\mathbf{B}_{k,\ell}$ ($\ell = 4$), block ternary keys $\mathbf{T}_{k,\ell}$ ($\ell = 4$) and key unrolling on $\mathbf{B}_{n/2,2}$ achieve a speedup of $1.12\times$ over the key unrolling method (CHES 2024) and $1.73\times$ over FINAL (ASIACRYPT 2021). Notably, the last 2 algorithms increase memory usage by less than 15MB compared to the key unrolling method (CHES 2024).

Keywords: Fully homomorphic encryption · Bootstrapping · NTRU
· Block key distribution · Key unrolling

1 Introduction

Fully homomorphic encryption (FHE) is critical for privacy-preserving computation, particularly in cloud computing and secure data processing [8,21]. An FHE scheme enables computations on ciphertexts without decryption. In 2009, Gentry’s breakthrough work [18] established the first FHE construction using ideal lattices and introduced the bootstrapping technique. Subsequently, various constructions have been developed, including BGV [7], B/FV [6,16], GSW [19] and CKKS [10], all of which are based on the learning with errors (LWE) [30] or Ring-LWE (RLWE) [28] problems. These schemes incorporate noise into ciphertexts, which accumulates over multiple operations and eventually renders

decryption unreliable. Bootstrapping addresses this limitation by reducing noise back to the initial level.

In 2014, Alperin-Sheriff and Peikert [2] introduced a new bootstrapping method that was significantly more efficient for ciphertexts encrypted from a single bit compared to the bootstrapping techniques for BGV [20] and CKKS [9]. This approach was implemented by Ducas et al. in 2015, resulting in the FHEW scheme [14]. Later, Gama et al. [17] proposed an optimization for special (LWE) private keys (e.g., $\mathbf{s} \in \{0, 1\}^n$), simplifying the blind rotation step, which is the most time-consuming part of bootstrapping. Chillotti et al. [11,12] further improved this using torus representations in TFHE. Bourse et al. [5] proposed the key unrolling technique to enhance the efficiency of the blind rotation in the GINX method [17].

In 2022, Bonte et al. [4] proposed using NTRU ciphertexts instead of RLWE ciphertexts in the blind rotation step. Owing to their simpler structure, NTRU ciphertexts can offer greater efficiency. Building upon this idea, Xiang et al. [33] optimized both the ciphertext structure and algorithm, enabling support for general secret keys. More recently, Li et al. [27] improved the blind rotation further by involving the key unrolling method [5] into the schemes in [4].

Despite these advances, the blind rotation step remains a significant bottleneck due to the large number of external product operations required. This motivates our exploration of methods to reduce the number of external products, thereby improving the overall efficiency of NTRU-based bootstrapping.

1.1 Our Contributions

This paper is devoted to optimizing the NTRU-based (GINX) blind rotation. More precisely, we propose three methods to reduce the number of external products:

- **Block binary key distribution.** Traditional GINX bootstrapping requires n external products of the form $\text{ACC} \leftarrow \text{ACC} + \text{ACC} \odot (Y^{a_i} - 1) \cdot \text{evk}_i$. Inspired by the work of Lee et al. [25], we adopt a block binary distribution $\mathbf{B}_{k,\ell}$ for LWE secret keys in which each group of ℓ components contains at most one non-zero entry. We extend this approach to NTRU-based bootstrapping, modifying the external product to the form

$$\text{ACC} \leftarrow \text{ACC} + \text{ACC} \odot \left[\sum_{i \in I_j} (Y^{a_i} - 1) \cdot \text{evk}_i \right].$$

Consequently, our method requires only n/ℓ external products, compared to n in [4].

- **Block ternary key distribution.** While a standard binary key space $\{0, 1\}^n$ contains 2^n elements, the block binary key space $\mathbf{B}_{k,\ell}$ has significantly fewer elements only $(\ell + 1)^{n/\ell}$ where $n = kl$. Thus, ℓ cannot be chosen too large without compromising security. In contrast, by adopting a block ternary distribution $\mathbf{T}_{k',\ell'}$ and employing the GINX bootstrapping method from [24], we

can achieve the same security level with a key space of size $(2^{\ell'} + 1)^{n/\ell'}$, while reducing the number of external products to n/ℓ' .

• **Combining block keys with key unrolling.** Another effective way to reduce the number of external products without sacrificing security is through the key unrolling technique introduced in [5]. Both [5] and [27] employ this method to halve the number of external products. When using a block binary key distribution $\mathbf{B}_{n/2,2}$, this combination enables to reduce the number of external products by a factor of four, to be $n/4$.

1.2 Related Work

Lee et al. [26] introduced a method based on generator elements and automorphisms, which significantly enhances the efficiency of the Alperin-SheriffPeikert (AP) bootstrapping method [2]. Building upon this, Wang et al. [31] further optimized the automorphism-based technique by reducing the number of required automorphisms, thereby improving the performances.

Bon et al. [3] proposed a novel technique for evaluating multiple logic gates using a single bootstrapping operation, which reduces computational overhead in logic circuit evaluations.

More recently, Corrigan-Gibbs et al. [13] introduced a new somewhat homomorphic encryption (SHE) scheme grounded in sparse learning parity with noise (LPN) and linear homomorphic encryption (LHE). Their approach leverages the sparsity of matrices to minimize the computational cost of homomorphic operations.

2 Preliminaries

2.1 Notations

We use $\lfloor \cdot \rfloor$, $\lceil \cdot \rceil$, $\text{round}(\cdot)$ to denote the floor, ceiling and round function on float numbers, respectively. Let $\mathbb{Z}$ be the ring of integers. For $q \in \mathbb{Z}$, denote $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ to be the residue ring modulo q . We use $[a]_q$ to represent $a \pmod{q}$ for some integer a . The arrow $\leftarrow$ is used to represent randomly chosen an element from a distribution. Denote the variance of a random variable a over $\mathbb{Z}_q$ by $\text{Var}(a)$. The error distribution χ used in this paper refers to the discrete Gaussian distribution over $\mathbb{Z}_q$ or $\mathbb{Z}_q^n$. A look up table (LUT) [12] means a map $h : \mathbb{Z}_t \rightarrow \mathbb{Z}_t$.

Let N be a power of 2. We use R to denote the $2N$ -th cyclotomic ring $\mathbb{Z}[X]/\langle X^N + 1 \rangle$. For a positive integer q , let R_q be the quotient ring R/qR . For $a \in R$, $\phi(a) = (a_0, a_1, \dots, a_{N-1})$ is the coefficient vector of a , and the variance of a is defined as $\text{Var}(a) = \text{Var}(\phi(a))$.

2.2 Gadget Decomposition

We describe a technique known as gadget decomposition [11], which is commonly used to reduce noise in the ciphertexts.

Lemma 1 (Gadget Decomposition) *For a fixed moduli Q and a decomposition base B , define the decomposition length as $d = \lceil \log_B Q \rceil$ and the gadget vector as $\mathbf{g} = (B^0, B^1, \dots, B^{d-1})$. Given a polynomial $a \in R_Q$, the signed decomposition in base B is defined as*

$$\mathbf{g}^{-1}(a) = \left([a]_B, \left\lfloor \left\lfloor \frac{a}{B} \right\rfloor \right\rfloor_B, \dots, \left\lfloor \left\lfloor \frac{a}{B^{d-1}} \right\rfloor \right\rfloor_B \right) \in R_B^d$$

where each component belongs to $[-B/2, B/2]$. It is easy to see that $\langle \mathbf{g}^{-1}(a), \mathbf{g} \rangle \equiv a \pmod{Q}$.

Remark 1. In our implementation (see Section 5), we employ an approximate variant of gadget decomposition [23,27] to improve computational efficiency.

2.3 Hard Problems and Ciphertexts

We recall some hard problems which will be used in this paper.

Definition 1 (Decisional LWE Problem [30]) *Suppose q and n are positive integers, and χ is the discrete Gaussian distribution over $\mathbb{Z}_q$. The decisional LWE problem is to distinguish between the distribution $(\mathbf{a}, -\langle \mathbf{a}, \mathbf{s} \rangle + e) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ and the uniform distribution over $\mathbb{Z}_q^{n+1}$, where $\mathbf{a} \in \mathbb{Z}_q^n$ is chosen uniformly at random, $e \leftarrow \chi$ and $\mathbf{s}$ is the secret key that is chosen from $\mathbb{Z}^n$.*

Definition 2 (Decisional NTRU Problem [22]) *Suppose Q and N are positive integers, χ is an error distribution over R . The decision NTRU problem is to distinguish between the distribution $g \cdot f \in R_Q$ and the uniform distribution over R_Q , where $g \leftarrow \chi$ is the noise polynomial, and the secret key f is invertible in R_Q .*

Definition 3 (Decisional Vectorized-NTRU Problem [33]) *Suppose Q and N are positive integers, χ is an error distribution over R . For a positive integer d , the decisional vectorized-NTRU problem is to distinguish between the distribution $\mathbf{g} \cdot f^{-1} \in R_Q^d$ and the uniform distribution over R_Q^d , where $\mathbf{g} \leftarrow \chi^d$ is the noise polynomial vector, and the secret key f is invertible in R_Q .*

For LWE ciphertexts, we use the B/FV-like ciphertexts [6,16]:

Definition 4 (LWE Ciphertexts) *Suppose q and n are positive integers, t is the moduli of the message space with $t \mid q$, and χ is an error distribution over $\mathbb{Z}_q$. The LWE ciphertext of a message $m \in \mathbb{Z}_t$ under the secret key $\mathbf{s} \in \mathbb{Z}_q^n$ is defined as*

$$\left(\mathbf{a}, b = -\langle \mathbf{a}, \mathbf{s} \rangle - \left\lfloor \frac{q}{t} \right\rfloor \cdot m + e \right) \in \mathbb{Z}_q^{n+1},$$

where $\mathbf{a} \leftarrow \mathbb{Z}_q^n$ and $e \leftarrow \chi$.

Next, we introduce the NTRU ciphertexts used in bootstrapping.

Definition 5 (NTRU Ciphertexts) *Let Q be a positive integer. The (scalar) NTRU ciphertext of a message polynomial $m \in R_Q$ is defined as*

$$\text{NTRU}_{f,Q}(m) = (g + m) \cdot f^{-1} \in R_Q,$$

where the error polynomial g is sampled from the error distribution χ , the secret key f is invertible in R_Q and the coefficients of f belong to $\{-1, 0, 1\}$.

The vector-NTRU Ciphertexts is defined as

$$\text{VNTRU}_{f,Q}(m) = \mathbf{g} \cdot f^{-1} + \mathbf{g} \cdot m,$$

where the error polynomial vector $\mathbf{g}$ is sampled from the error distribution χ^d with $d = \lceil \log_B Q \rceil$, the gadget vector $\mathbf{g}$ is defined as in Lemma 1, and the secret key f satisfies the same conditions with these in the NTRU ciphertexts.

Lemma 2 (External Product [33]) *Let $c = \text{NTRU}_{Q,f}(u) \in R_Q$ be an NTRU ciphertext with noise variance $\text{Var}(c)$. Let $\mathbf{c}' = \text{VNTRU}_{Q,f}(v)$ be a vector-NTRU ciphertext with noise variance $\text{Var}(\mathbf{c}')$. Define the external product $\odot$ as*

$$c \odot \mathbf{c}' = \langle \mathbf{g}^{-1}(c), \mathbf{c}' \rangle := \hat{c}.$$

Then we have $\hat{c} = (\hat{g} + u \cdot v) \cdot f^{-1}$ is an NTRU ciphertext for $u \cdot v$ (i.e., $\hat{c} = \text{NTRU}_{Q,f}(u \cdot v)$), whose variance satisfies

$$\text{Var}(\hat{c}) \leq d \cdot N \cdot \frac{B^2}{12} \cdot \text{Var}(\mathbf{c}') + \|v\|_2^2 \cdot \text{Var}(c).$$

2.4 Bootstrapping

We firstly recall several results that play key roles in the bootstrapping procedure.

Lemma 3 (Sample Extraction [27,33]) *Assume we have an NTRU ciphertext*

$$c = \text{NTRU}_{f,Q}(u) = (g + u) \cdot f^{-1} \in R_Q, \text{ where } c = \sum_{i=0}^{N-1} c_i \cdot X^i.$$

Then the sample extraction algorithm can compute

$$\text{Ext}(c) = (c_0, -c_{N-1}, -c_{N-2}, \dots, -c_1, 0)$$

and we have $\text{Ext}(c) \in \text{LWE}_{\phi(f),Q}^N(m_0)$.

Lemma 4 (Key Switching [7,6]) *Let Q_k, B_k be two integers and $d_k = \lceil \log_{B_k} Q_k \rceil$. Assume we have an LWE ciphertext $\text{ct} = (\mathbf{a}, b) \in \text{LWE}_{\mathbf{s},Q_k}^N(m)$ with noise variance $\text{Var}(\text{ct})$, which generates the switching keys $\mathbf{ksk}_{i,j,v} \in \text{LWE}_{\mathbf{s},Q_k}^n(v f_i B_k^j)$,*

where $0 \leq v \leq B_k - 1$, $0 \leq i \leq N - 1$, $0 \leq j \leq d_k - 1$, having noise variance $\text{Var}(\mathbf{k}\mathbf{s}\mathbf{k})$. Then the LWE key switching algorithm computes

$$\text{KeySwitch}(\text{ct}) = (0, b) - \sum_{i,j} \mathbf{k}\mathbf{s}\mathbf{k}_{i,j,a_{i,j}},$$

and we have $\text{ct}' = \text{KeySwitch}(\text{ct}) \in \text{LWE}_{\mathbf{s}, Q_k}^n(m)$, whose variance satisfies

$$\text{Var}(\text{ct}') \leq \text{Var}(\text{ct}) + N \cdot d_k \cdot \text{Var}(\mathbf{k}\mathbf{s}\mathbf{k}).$$

Lemma 5 (Modulus Switching [7,6]) Assume we have an LWE ciphertext $\text{ct} = (\mathbf{a}, b) \in \text{LWE}_{\mathbf{s}, Q}^n(m)$ with error variance $\text{Var}(\text{ct})$. Then the modulus switching algorithm computes

$$\text{ModSwitch}(\text{ct}) = \left(\left\lfloor \frac{q}{Q} \cdot \mathbf{a} \right\rfloor, \left\lfloor \frac{q}{Q} \cdot b \right\rfloor \right),$$

and we have $\text{ct}' = \text{ModSwitch}(\text{ct}) \in \text{LWE}_{\mathbf{s}, q}^n(m)$, whose variance satisfies

$$\text{Var}(\text{ct}') \leq \frac{q^2}{Q^2} \cdot \text{Var}(\text{ct}) + \frac{n \cdot \text{Var}(\mathbf{s}) + 1}{12}.$$

The purpose of bootstrapping is to refresh the noise of a ciphertext to the initial level. Chilotti et al [12] proposed the functional bootstrapping which can compute an LUT on the plaintext when applying the bootstrapping algorithm. The functional bootstrapping algorithm consists of five steps: Initialization, Blind Rotation, Sample Extration, Key Switching and Modulus Switching. Figure 1 illustrates the detailed process of NTRU-based functional bootstrapping.

Initialization. For an input LWE ciphertext $(\mathbf{a}, b) \in \text{LWE}_{\mathbf{s}, q}^n(m)$, compute the test polynomial $P(X)$ and output an NTRU ciphertext $\text{NTRU}_{f, Q}(P(X) \cdot X^{\frac{2N}{q}} \cdot b)$;

Blind Rotation. Input the first member $\mathbf{a}$ of the LWE ciphertext $(\mathbf{a}, b)$ and $\text{NTRU}_{f, Q}(P(X) \cdot X^{\frac{2N}{q}} \cdot b)$. Use external products to homomorphically multiply $X^{\frac{2N}{q} \cdot \sum_{i=0}^{n-1} a_i s_i}$ and output $c = \text{NTRU}_{f, Q}\left(P(X) \cdot X^{\frac{2N}{q} \cdot (b + \sum_{i=0}^{n-1} a_i s_i)}\right)$;

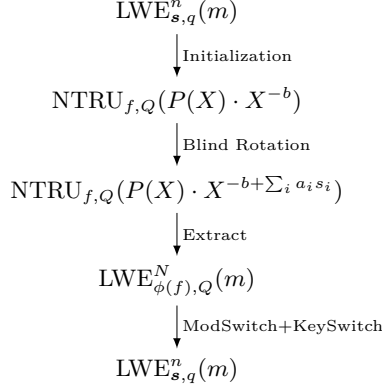
Sample Extration. Input the NTRU ciphertext $c = \text{NTRU}_{f, Q}(u) = \sum_{i=0}^{N-1} c_i \cdot X^i$ and output $(\mathbf{a}, b) \in \text{LWE}_{\phi(f), Q}^N(m)$ as in Lemma 3;

Key Switching. Input $(\mathbf{a}, b) \in \text{LWE}_{\phi(f), Q}^N(m)$ and output $(\mathbf{a}', b') \in \text{LWE}_{\mathbf{s}, Q}^n(m)$ as in Lemma 4;

Modulus Switching. Input $(\mathbf{a}', b') \in \text{LWE}_{\mathbf{s}, Q}^n(m)$ and output $(\mathbf{a}', b') \in \text{LWE}_{\mathbf{s}, q}^n(m)$ as in Lemma 5.

3 Faster Bootstrapping Using Block Keys

In this section, we use block keys to improve the efficiency of NTRU-based bootstrapping algorithm.

**Fig. 1.** NTRU-based Functional Bootstrapping

Firstly, we introduce the definition of block binary distribution $\mathbf{B}_{k,\ell}$. Then we use the property of block keys to modify the blind rotation algorithm. It turns out that we can decrease the number of external products, which is the most time-consuming step.

Since the set of block binary keys has less elements than the set of general binary keys, namely, $\{0, 1\}^n$, we also provide the method of block ternary distribution $\mathbf{T}_{k,\ell}$. It can simply decrease the number of external products without reduce the size of key space too much.

3.1 Block Binary Keys

Definition 6 (Block Binary Distribution [25]) *Let k, ℓ be positive integers and $n = k \cdot \ell$. Let $\mathbf{B}_\ell$ be the distribution over $\mathbb{Z}^\ell$ from which we can sample the vectors $(1, \dots, 0), (0, 1, \dots, 0), \dots, (0, 0, \dots, 1), (0, \dots, 0)$ with equal probability. The block binary distribution $\mathbf{B}_{k,\ell}$ is a distribution on $\mathbb{Z}^n$, from which we can sample $(\mathbf{a}_1, \dots, \mathbf{a}_k) \in \mathbb{Z}^n$, where $\mathbf{a}_i \leftarrow \mathbf{B}_\ell$ for $1 \leq i \leq k$.*

Let $I_j = \{j\ell, j\ell + 1, \dots, j(\ell + 1) - 1\}$, $0 \leq j < k$. If the LWE private key $\mathbf{s} = (s_0, \dots, s_{n-1}) \leftarrow \mathbf{B}_{k,\ell}$, then we have

$$Y^{\sum_{i \in I_j} a_i s_i} = 1 + \sum_{i \in I_j} (Y^{a_i} - 1) s_i.$$

Thus our external-product step can be improved as

$$\text{ACC} \leftarrow \text{ACC} + \text{ACC} \odot \left(\sum_{i \in I_j} (Y^{a_i} - 1) \cdot \text{ev}\mathbf{k}_i \right) \text{ for } j = 0, 1, \dots, k - 1.$$

The detailed algorithm is described in Algorithm 1. Theorem 1 shows the correctness of Algorithm 1 and gives the noise variance.

Algorithm 1 Revised NTRU Bootstrapping Using Block Binary Keys

Input: An LWE sample $\text{ct} = \langle \mathbf{a}, b = -\langle \mathbf{a}, \mathbf{s} \rangle - \lfloor \frac{q}{t} \rfloor \cdot m + e \rangle \in \text{LWE}_{\mathbf{s},q}^n(m)$.

The bootstrapping key $\{\mathbf{evk}_i\}_{i=0}^{n-1}$.

A special bootstrapping key $\mathbf{evk}' = \text{VNTRU}_{f,Q}(f^{-1})$.

An LWE key switching key $\mathbf{ksk}_{\mathbf{s},(\phi(h))}$.

Output: An LWE sample $\text{ct}' \in \text{LWE}_{\mathbf{s},q}^n(h(m))$.

1: Set $P(X) = \sum_{i=0}^{N-1} \frac{Q}{t} \cdot h\left(\left\lfloor \frac{t}{q} \cdot i \right\rfloor\right) \cdot X^i$, and $Y = X^{\frac{2N}{q}}$

2: Let $\text{ACC} = (P(X) \cdot Y^b) \odot \mathbf{evk}'$

for $j = 0$; $j < k$; $j = j + 1$ **do**

$$\text{ACC} \leftarrow \text{ACC} + \text{ACC} \odot \left(\sum_{i \in I_j} (Y^{a_i} - 1) \cdot \mathbf{evk}_i \right).$$

end for

3: $\text{ct}' \leftarrow \text{Ext}(\text{ACC})$

4: $\text{ct}' \leftarrow \text{ModSwitch}_{Q \rightarrow Q_k}(\text{ct}')$

5: $\text{ct}' \leftarrow \text{LWE.KeySwitch}(\text{ct}')$

6: $\text{ct}' \leftarrow \text{ModSwitch}_{Q_k \rightarrow q}(\text{ct}')$

return ct'

Theorem 1 *let $h : \mathbb{Z}_t \rightarrow \mathbb{Z}_t$ be an LUT, $\text{ct} \in \text{LWE}_{\mathbf{s},q}^n(m)$ be an LWE ciphertext where $\mathbf{s} \leftarrow \mathbf{B}_{k,\ell}$, and σ^2 be the variance of the polynomial error distribution. Algorithm 1 will output an LWE ciphertext $\text{ct}' \in \text{LWE}_{\mathbf{s},q}^n(h(m))$, satisfying*

$$\text{Var}(\text{ct}') \leq \frac{q^2}{Q_k^2} \cdot \left(\frac{Q_k^2}{Q^2} \cdot \sigma_1^2 + \sigma_2^2 + \sigma_3^2 \right) + \sigma_4^2,$$

where $\sigma_1^2 = (2n+1) \cdot d \cdot N \cdot \frac{B^2}{12} \cdot \sigma^2$, $\sigma_2^2 = \frac{N \cdot \text{Var}(\phi(f)) + 1}{12}$, $\sigma_3^2 = N \cdot d_k \cdot \text{Var}(\mathbf{ksk})$, $\sigma_4^2 = \frac{n \cdot \text{Var}(\mathbf{s}) + 1}{12}$.

Proof. Let $\mathbf{evk}' = (\mathbf{g} + \mathbf{g}') \cdot f^{-1}$, $\mathbf{g} = (g_0, g_1, \dots, g_{d-1})$, $\mathbf{g}_A^{-1}(P(X) \cdot Y^b) = (c_0, c_1, \dots, c_{d-1})$. After Line 2 in Algorithm 1, we have

$$\begin{aligned} \text{ACC} &= (P(X) \cdot Y^b) \odot \mathbf{evk}' \\ &= \langle \mathbf{g}_A^{-1}(P(X) \cdot Y^b), (\mathbf{g} + \mathbf{g}') \cdot f^{-1} \rangle \\ &= \left(\sum_{i=0}^{d-1} c_i \cdot g_i + P(X) \cdot Y^b \right) \cdot f^{-1} \\ &= \text{NTRU}_{f,Q}(P(X) \cdot Y^b). \end{aligned}$$

For $i = 0$, we have

$$\text{ACC} + \text{ACC} \odot \left(\sum_{i \in I_j} (Y^{a_i} - 1) \cdot \mathbf{evk}_i \right) = \text{NTRU}_{f,Q} \left(P(X) \cdot Y^b \cdot Y^{\sum_{i \in I_0} a_i \cdot s_i} \right).$$

Thus after k steps, we have

$$\begin{aligned}\text{ACC} &= \text{NTRU}_{f,Q} \left(P(X) \cdot Y^b \cdot Y^{\sum_{j=0}^{k-1} \sum_{i \in I_j} a_i \cdot s_i} \right) \\ &= \text{NTRU}_{f,Q} \left(P(X) \cdot Y^b \cdot Y^{\sum_{i=0}^{n-1} a_i \cdot s_i} \right) \\ &= \text{NTRU}_{f,Q} \left(P(X) \cdot Y^{-\lfloor \frac{q}{t} \rfloor \cdot m + e} \right).\end{aligned}$$

Since $P(X) = \sum_{i=0}^{N-1} \frac{Q}{t} \cdot h \left(\left\lfloor \frac{t}{q} \cdot i \right\rfloor \right) \cdot X^i$, by Lemma 3 we have $\text{Ext}(\text{ACC}) \in \text{LWE}_{f,Q}^N(h(m))$. Then performing Line 4–6, by Lemma 4 and Lemma 5, we get $\text{ct}' \in \text{LWE}_{\mathbf{s},q}^n(h(m))$.

As for the noise condition, note that after Line 2, we perform one external product. By Lemma 2, we have $\text{Var}(\text{ACC}) \leq d \cdot N \cdot \frac{B^2}{12} \cdot \sigma^2$. For each external product step, since $\text{ACC} = \text{NTRU}_{f,Q}(u)$ for some u , we have

$$\begin{aligned}\text{ACC}_{\text{new}} &= \text{ACC} + \text{ACC} \odot \left(\sum_{i \in I_j} (Y^{a_i} - 1) \cdot \mathbf{evk}_i \right) \\ &= \text{ACC} + \left\langle \mathbf{g}^{-1}(\text{ACC}), \sum_{i \in I_j} (Y^{a_i} - 1) \cdot s_i \cdot \mathbf{g} \right\rangle + \left\langle \mathbf{g}^{-1}(\text{ACC}), \sum_{i \in I_j} (Y^{a_i} - 1) \cdot \mathbf{g}_i \cdot f^{-1} \right\rangle.\end{aligned}$$

Thus we have

$$\text{Var}(\text{ACC}_{\text{new}}) \leq \text{Var}(\text{ACC}) + 2 \cdot \ell \cdot d \cdot N \cdot \frac{B^2}{12} \cdot \sigma^2,$$

since

$$\text{Var} \left(\left\langle \mathbf{g}^{-1}(\text{ACC}), \sum_{i \in I_j} (Y^{a_i} - 1) \cdot \mathbf{g}_i \cdot f^{-1} \right\rangle \right) \leq \ell \cdot \|Y^{a_i} - 1\|_2^2 \cdot d \cdot N \cdot \frac{B^2}{12} \cdot \sigma^2.$$

After k external products, the variance satisfies

$$\text{Var}(\text{ACC}) \leq (2n + 1) \cdot d \cdot N \cdot \frac{B^2}{12} \cdot \sigma^2.$$

Then the conclusion can be obtained by Lemma 3, Lemma 4 and Lemma 5. $\square$

Remark 2. Since the cardinality of $\mathbf{B}_{k,\ell}$ is significantly smaller than that of the full binary key space $\{0, 1\}^n$, a larger dimension n may be required to achieve the same level of security.

3.2 Block Ternary Keys

Before introducing our block ternary key approach, we briefly review the GINX method for ternary secret keys; see [24].

Bootstrapping for Ternary keys Let $\mathbf{s} = (s_0, \dots, s_{n-1}) \leftarrow \{0, 1, -1\}^n$. We can use 2 vectors $s^1 = (s_0^1, \dots, s_{n-1}^1), s^2 = (s_0^2, \dots, s_{n-1}^2) \in \mathbb{B}^n$ to denote $\mathbf{s}$, i.e.,

$$(s_0, \dots, s_{n-1}) = (s_0^1, \dots, s_{n-1}^1) - (s_0^2, \dots, s_{n-1}^2), \text{ where}$$

$$(s_i^1, s_i^2) = \begin{cases} (0, 0) & \text{if } s_i = 0; \\ (1, 0) & \text{if } s_i = 1; \\ (0, 1) & \text{if } s_i = -1 \end{cases} \text{ for } 0 \leq i \leq n-1.$$

If we compute, for $0 \leq i \leq n-1$, that

$$\mathbf{evk}_{2i} = \text{NTRU}'_{Q,f,\tau}(s_i^1), \quad \mathbf{evk}_{2i+1} = \text{NTRU}'_{Q,f,\tau}(s_i^2),$$

then the external product step can be expressed as

$$\text{ACC} \leftarrow \text{ACC} + \text{ACC} \odot [(Y^{a_i} - 1) \cdot \mathbf{evk}_{2i} + (Y^{-a_i} - 1) \cdot \mathbf{evk}_{2i+1}].$$

We now extend this to block ternary keys to improve the efficiency of bootstrapping. First we give the definition of block ternary distribution.

Definition 7 (Block Ternary Distribution) Let k, ℓ be positive integers and $n = k \cdot \ell$. Let $\mathbf{T}_\ell$ be a distribution over $\mathbb{Z}^\ell$ from which we can sample the vectors $(1, \dots, 0), (0, 1, \dots, 0), \dots, (0, 0, \dots, 1), (-1, \dots, 0), (0, -1, \dots, 0), \dots, (0, 0, \dots, -1), (0, \dots, 0)$ with equal probability. Define the block ternary distribution $\mathbf{T}_{k,\ell}$ as a distribution over $\mathbb{Z}^n$, from which we can sample $(\mathbf{a}_1, \dots, \mathbf{a}_k) \in \mathbb{Z}^n$ where $\mathbf{a}_i \leftarrow \mathbf{T}_\ell$ for $1 \leq i \leq k$.

Our block ternary key bootstrapping algorithm is presented in Algorithm 2. One can observe that this algorithm needs only to execute n/ℓ external products. However, the bootstrapping algorithm for ternary key distribution is more complicated. We should memorize more bootstrapping keys than using $\mathbf{B}_{k,\ell}$. Theorem 2 shows correctness of Algorithm 2 and gives the noise variance.

Theorem 2 Let $h : \mathbb{Z}_t \rightarrow \mathbb{Z}_t$ be an LUT, $\text{ct} \in \text{LWE}_{\mathbf{s},q}^n(m)$ be an LWE ciphertext where $\mathbf{s} \leftarrow \mathbf{T}_{k,\ell}$, and σ^2 be the variance of polynomial error distribution. Algorithm 2 will output an LWE ciphertext $\text{ct}' \in \text{LWE}_{\mathbf{s},q}^n(h(m))$ satisfying

$$\text{Var}(\text{ct}') \leq \frac{q^2}{Q_k^2} \cdot \left(\frac{Q_k^2}{Q^2} \cdot \sigma_1^2 + \sigma_2^2 + \sigma_3^2 \right) + \sigma_4^2,$$

where $\sigma_1^2 = (2n+1) \cdot d \cdot N \cdot \frac{B^2}{12} \cdot \sigma^2$, $\sigma_2^2 = \frac{N \cdot \text{Var}(\phi(f)) + 1}{12}$, $\sigma_3^2 = N \cdot d_k \cdot \text{Var}(\mathbf{k} \mathbf{s} \mathbf{k})$ and $\sigma_4^2 = \frac{n \cdot \text{Var}(\mathbf{s}) + 1}{12}$.

Proof. The proof is very similar to that of **Theorem 1**, so we omit it here. $\square$

Remark 3. Note that the cardinality of $\mathbf{B}_{k,\ell}$ is $(\ell+1)^{\frac{n}{\ell}}$, while that of $\mathbf{T}_{k',\ell'}$ is $(2\ell'+1)^{\frac{n}{\ell'}}$. Hence, we can choose a larger $\ell' > \ell$ without compromising security. For instance, setting $\ell = 2$, $\ell' = 4$, we have $\#\mathbf{B}_{k,\ell} = \#\mathbf{T}_{k',\ell'}$, but the ternary approach requires only $n/4$ external products compared to $n/2$ in the binary case.

Algorithm 2 Revised NTRU Bootstrapping using Block Ternary Keys

Input: An LWE sample $\text{ct} = (\mathbf{a}, b = -\langle \mathbf{a}, \mathbf{s} \rangle - \lfloor \frac{q}{t} \rfloor \cdot m + e) \in \text{LWE}_{\mathbf{s}, q}^n(m)$.

The bootstrapping key $\{\mathbf{evk}_i\}_{i=0}^{2n-1}$.

A special bootstrapping key $\mathbf{evk}' = \text{VNTRU}_{f, Q}(f^{-1})$.

An LWE key switching key $\mathbf{ksk}_{\mathbf{s}, (\phi(h))}$.

Output: An LWE sample $\text{ct}' \in \text{LWE}_{\mathbf{s}, q}^n(h(m))$.

1: Set $P(X) = \sum_{i=0}^{N-1} \frac{Q}{t} \cdot h\left(\left\lfloor \frac{t}{q} \cdot i \right\rfloor\right) \cdot X^i$, and $Y = X^{\frac{2N}{q}}$

2: Let $\text{ACC} = (P(X) \cdot Y^b) \odot \mathbf{evk}'$

for $j = 0; j < k; j = j + 1$ **do**

$$\text{ACC} \leftarrow \text{ACC} + \text{ACC} \odot \left[\sum_{i \in I_j} (Y^{a_i} - 1) \cdot \mathbf{evk}_{2i} + \sum_{i \in I_j} (Y^{-a_i} - 1) \cdot \mathbf{evk}_{2i+1} \right].$$

end for

3: $\text{ct}' \leftarrow \text{Ext}(\text{ACC})$

4: $\text{ct}' \leftarrow \text{ModSwitch}_{Q \rightarrow Q_k}(\text{ct}')$

5: $\text{ct}' \leftarrow \text{LWE.KeySwitch}(\text{ct}')$

6: $\text{ct}' \leftarrow \text{ModSwitch}_{Q_k \rightarrow q}(\text{ct}')$

return ct'

4 Key Unrolling and Comparison

In this section, we introduce another technique to enhance bootstrapping efficiency, that is, the key unrolling, and compare the computational and memory costs of various methods.

4.1 Key Unrolling

For $Y = X^{2N/q}$, our goal is to homomorphically compute $Y^{b + \sum_{i=0}^{n-1} a_i s_i}$. Suppose the secret key $\mathbf{s} \leftarrow \mathbf{B}_{n/2, 2}$, then we can use 1 external product to homomorphically multiply

$$Y^{a_{4i} s_{4i} + a_{4i+1} s_{4i+1} + a_{4i+2} s_{4i+2} + a_{4i+3} s_{4i+3}}.$$

Since $(s_{4i}, s_{4i+1}) \neq (1, 1)$, $s_{4i+2}, s_{4i+3} \neq (1, 1)$, then we have

$$\begin{aligned} Y^{a_{4i} s_{4i} + a_{4i+1} s_{4i+1} + a_{4i+2} s_{4i+2} + a_{4i+3} s_{4i+3}} &= Y^{a_{4i} + a_{4i+2}} s_{4i} s_{4i+2} (1 - s_{4i+1}) (1 - s_{4i+3}) \\ &+ Y^{a_{4i} + a_{4i+3}} s_{4i} s_{4i+3} (1 - s_{4i+1}) (1 - s_{4i+2}) + Y^{a_{4i+1} + a_{4i+2}} s_{4i+1} s_{4i+2} (1 - s_{4i}) (1 - s_{4i+3}) \\ &+ Y^{a_{4i+1} + a_{4i+3}} s_{4i+1} s_{4i+3} (1 - s_{4i}) (1 - s_{4i+2}) + Y^{a_{4i}} s_{4i} (1 - s_{4i+1}) (1 - s_{4i+2}) (1 - s_{4i+3}) \\ &+ Y^{a_{4i+1}} s_{4i+1} (1 - s_{4i}) (1 - s_{4i+2}) (1 - s_{4i+3}) + Y^{a_{4i+2}} s_{4i+2} (1 - s_{4i+1}) (1 - s_{4i}) (1 - s_{4i+3}) \\ &+ Y^{a_{4i+3}} s_{4i+3} (1 - s_{4i+1}) (1 - s_{4i+2}) (1 - s_{4i}) \end{aligned} \quad (1)$$

$$\begin{aligned}
&= (Y^{a_{4i}+a_{4i+2}} - 1)s_{4i}s_{4i+2}(1-s_{4i+1})(1-s_{4i+3}) \\
&+ (Y^{a_{4i}+a_{4i+3}} - 1)s_{4i}s_{4i+3}(1-s_{4i+1})(1-s_{4i+2}) \\
&+ (Y^{a_{4i+1}+a_{4i+2}} - 1)s_{4i+1}s_{4i+2}(1-s_{4i})(1-s_{4i+3}) \\
&+ (Y^{a_{4i+1}+a_{4i+3}} - 1)s_{4i+1}s_{4i+3}(1-s_{4i})(1-s_{4i+2}) \\
&+ (Y^{a_{4i}} - 1)s_{4i}(1-s_{4i+1})(1-s_{4i+2})(1-s_{4i+3}) \\
&+ (Y^{a_{4i+1}} - 1)s_{4i+1}(1-s_{4i})(1-s_{4i+2})(1-s_{4i+3}) \\
&+ (Y^{a_{4i+2}} - 1)s_{4i+2}(1-s_{4i+1})(1-s_{4i})(1-s_{4i+3}) \\
&+ (Y^{a_{4i+3}} - 1)s_{4i+3}(1-s_{4i+1})(1-s_{4i+2})(1-s_{4i}) + 1.
\end{aligned} \tag{2}$$

For each $0 \leq i \leq \frac{n}{4}-1$, There are 9 valid combinations of $s_{4i}, s_{4i+1}, s_{4i+2}, s_{4i+3}$ listed in Table 1. For each combination, we precompute 8 evaluation keys $\mathbf{evk}_{i,j}$ ($0 \leq j \leq 7$) = $\text{VNTRU}_{f,q}(t_{i,j})$, where the values of $t_{i,j}$ are specified in Table 2.

Table 1. Case for $s_{4i+j}, j = 0, 1, 2, 3$.

Case	s_{4i}	s_{4i+1}	s_{4i+2}	s_{4i+3}
1	1	0	1	0
2	1	0	0	1
3	0	1	1	0
4	0	1	0	1
5	1	0	0	0
6	0	1	0	0
7	0	0	1	0
8	0	0	0	1
9	0	0	0	0

Each external product step then becomes

$$\begin{aligned}
\text{ACC} &= \text{ACC} + (Y^{a_{4i}+a_{4i+2}} - 1) \cdot \mathbf{evk}_{i,0} + (Y^{a_{4i}+a_{4i+3}} - 1) \cdot \mathbf{evk}_{i,1} \\
&+ (Y^{a_{4i+1}+a_{4i+2}} - 1) \cdot \mathbf{evk}_{i,2} + (Y^{a_{4i+1}+a_{4i+3}} - 1) \cdot \mathbf{evk}_{i,3} + (Y^{a_{4i}} - 1) \cdot \mathbf{evk}_{i,4} \\
&+ (Y^{a_{4i+1}} - 1) \cdot \mathbf{evk}_{i,5} + (Y^{a_{4i+2}} - 1) \cdot \mathbf{evk}_{i,6} + (Y^{a_{4i+3}} - 1) \cdot \mathbf{evk}_{i,7} \odot \text{ACC}.
\end{aligned}$$

We give the detailed bootstrapping algorithm in Algorithm 3. Since we need to memorize 8 bootstrapping keys for each $0 \leq i \leq n/4-1$, we need to memory $2n$ bootstrapping keys which is 33% higher than [27]. Theorem 3 shows the correctness of Algorithm 3 and gives the noise variance.

Theorem 3 Let $h : \mathbb{Z}_t \rightarrow \mathbb{Z}_t$ be an LUT, $\text{ct} \in \text{LWE}_{\mathbf{s},q}^n(m)$ be an LWE ciphertext where $\mathbf{s} \leftarrow \mathbf{B}_{n/2,2}$, and σ^2 be the variance of polynomial error distribution. Algorithm 3 will output an LWE ciphertext $\text{ct}' \in \text{LWE}_{\mathbf{s},q}^n(h(m))$ satisfying

$$\text{Var}(\text{ct}') \leq \frac{q^2}{Q_k^2} \cdot \left(\frac{Q_k^2}{Q^2} \cdot \sigma_1^2 + \sigma_2^2 + \sigma_3^2 \right) + \sigma_4^2,$$

Table 2. Values for $t_{i,j}$.

Case	$t_{i,0}$	$t_{i,1}$	$t_{i,2}$	$t_{i,3}$	$t_{i,4}$	$t_{i,5}$	$t_{i,6}$	$t_{i,7}$
1	1	0	0	0	0	0	0	0
2	0	1	0	0	0	0	0	0
3	0	0	1	0	0	0	0	0
4	0	0	0	1	0	0	0	0
5	0	0	0	0	1	0	0	0
6	0	0	0	0	0	1	0	0
7	0	0	0	0	0	0	1	0
8	0	0	0	0	0	0	0	1
9	0	0	0	0	0	0	0	0

where $\sigma_1^2 = (2n+1) \cdot d \cdot N \cdot \frac{B^2}{12} \cdot \sigma^2$, $\sigma_2^2 = \frac{N \cdot \text{Var}(\phi(f)) + 1}{12}$, $\sigma_3^2 = N \cdot d_k \cdot \text{Var}(\mathbf{k} \mathbf{s} \mathbf{k})$ and $\sigma_4^2 = \frac{n \cdot \text{Var}(\mathbf{s}) + 1}{12}$.

Proof. The proof is very similar to that of **Theorem 1**, so we omit it here. $\square$

Remark 4. For general ℓ , if $\mathbf{s} \leftarrow \mathbf{B}_{k,\ell}$, we have

$$\begin{aligned}
& Y^{\sum_{i \in I_j} a_i s_i + \sum_{i \in I_{j+1}} a_i s_i} \\
&= 1 + \sum_{a_i \in I_j \cup I_{j+1}} \left[(Y^{a_i} - 1) s_i \cdot \prod_{t \in I_j \cup I_{j+1} - \{i\}} (1 - s_t) \right] \\
&+ \sum_{\substack{i_1 \in I_j, \\ i_2 \in I_{j+1}}} \left[(Y^{a_{i_1} + a_{i_2}} - 1) s_{i_1} s_{i_2} \cdot \prod_{\substack{t \in I_j \cup I_{j+1} \\ - \{i_1, i_2\}}} (1 - s_t) \right].
\end{aligned} \tag{3}$$

Thus we may use 1 external product to homomorphically multiply 2ℓ terms, i.e., the total number of external product will be $n/2\ell$.

However, the RHS of (3) has $\ell^2 + 2\ell$ terms (except the term "1"). Thus each step will memory $\ell^2 + 2\ell$ evaluation keys. As a result, we will generate $n(\ell + 2)$ evaluation keys, leading to a very large key space.

4.2 Comparison

In this subsection, we compare the computational cost and memory usage of the various bootstrapping algorithms discussed above. For computation, we measure the number of number theoretic transforms (NTTs) and Hadamard multiplications. For memory, we count the number of vector-NTRU ciphertexts stored.

Algorithm 3 Revised NTRU Bootstrapping Using Block Binary Keys and Key Unrolling

Input: An LWE sample $\text{ct} = (\mathbf{a}, b = -\langle \mathbf{a}, \mathbf{s} \rangle - \lfloor \frac{q}{t} \rfloor \cdot m + e) \in \text{LWE}_{\mathbf{s},q}^n(m)$.

The bootstrapping key $\{\mathbf{evk}_{i,j}\}, 0 \leq i \leq n/4, 0 \leq j \leq 7$.

A special bootstrapping key $\mathbf{evk}' = \text{VNTRU}_{f,Q}(f^{-1})$.

An LWE key switching key $\mathbf{ksk}_{\mathbf{s},(\phi(h))}$.

Output: An LWE sample $\text{ct}' \in \text{LWE}_{\mathbf{s},q}^n(h(m))$.

1: Set $P(X) = \sum_{i=0}^{N-1} \frac{Q}{t} \cdot h\left(\lfloor \frac{t}{q} \cdot i \rfloor\right) \cdot X^i$, and $Y = X^{\frac{2N}{q}}$

2: Let $\text{ACC} = (P(X) \cdot Y^b) \odot \mathbf{evk}'$

for $i = 0; i < n/4; i = i + 1$ **do**

$$\begin{aligned} \text{ACC} = & \text{ACC} + \text{ACC} \odot ((Y^{a_{4i}+a_{4i+2}} - 1) \cdot \mathbf{evk}_{i,0} + (Y^{a_{4i}+a_{4i+3}} - 1) \cdot \mathbf{evk}_{i,1} \\ & + (Y^{a_{4i+1}+a_{4i+2}} - 1) \cdot \mathbf{evk}_{i,2} + (Y^{a_{4i+3}+a_{4i+2}} - 1) \cdot \mathbf{evk}_{i,3} + (Y^{a_{4i}} - 1) \cdot \mathbf{evk}_{i,4} \\ & + (Y^{a_{4i+1}} - 1) \cdot \mathbf{evk}_{i,5} + (Y^{a_{4i+2}} - 1) \cdot \mathbf{evk}_{i,6} + (Y^{a_{4i+3}} - 1) \cdot \mathbf{evk}_{i,7}). \end{aligned}$$

end for

3: $\text{ct}' \leftarrow \text{Ext}(\text{ACC})$

4: $\text{ct}' \leftarrow \text{ModSwitch}_{Q \rightarrow Q_k}(\text{ct}')$

5: $\text{ct}' \leftarrow \text{LWE.KeySwitch}(\text{ct}')$

6: $\text{ct}' \leftarrow \text{ModSwitch}_{Q_k \rightarrow q}(\text{ct}')$

return ct'

We begin with a detailed analysis of Algorithm 1 (block binary keys). Prior to the blind rotation step, we precompute the NTT representations of

- The test polynomial $P(X)$;
- All powers Y^i and $Y^i - 1$ for $0 \leq i < q$;
- All bootstrapping keys $\mathbf{evk}_i$ (or $\mathbf{evk}_{i,j}$ for extended methods).

Initial External Product. To compute $P(X) \cdot Y^b \odot \text{VNTRU}_{f,Q}(f^{-1})$, we perform

- One Hadamard multiplication for $P(X) \cdot Y^b$;
- d NTTs compute the NTT representations of $g^{-1}(P(X) \cdot Y^b)$;
- d Hadamard multiplications for the (polynomial) inner product;
- One INTT to return to coefficient representation.

Hence, this step costs $(d+1)$ NTTs and $(d+1)$ Hadamard multiplications.

Blind Rotation. Each blind rotation iteration requires

- d NTTs to compute the NTT representations of $g^{-1}(\text{ACC})$;
- d Hadamard multiplications for each (polynomial) inner product;
- One Hadamard multiplication for each $(Y^{a_i} - 1) \cdot \langle g^{-1}(\text{ACC}), \mathbf{evk}_i \rangle$;
- One INTT to return to coefficient representation.

Total per iteration: $(d+1)$ NTTs and $\ell \cdot (d+1)$ Hadamard multiplications.

The same analysis applies to Algorithm 2 and Algorithm 3, with variations in the number of blocks and keys. Table 3 shows the results.

Table 3. Computational cost and size of memory for different algorithms

Method	# NTT	# Hadamard Mult	#Memory
[4]	$n \cdot (d + 1)$	$n \cdot (d + 1)$	n
[27]	$(1 + \frac{n}{2}) \cdot (d + 1)$	$(\frac{3n}{2} + 1) \cdot (d + 1)$	$\frac{3n}{2} + 1$
Block Binary ($\ell = 2$)	$(1 + \frac{n}{2}) \cdot (d + 1)$	$(n + 1) \cdot (d + 1)$	$n + 1$
Block Binary ($\ell = 3$)	$(1 + \frac{n}{3}) \cdot (d + 1)$	$(n + 1) \cdot (d + 1)$	$n + 1$
Block Binary ($\ell = 4$)	$(1 + \frac{n}{4}) \cdot (d + 1)$	$(n + 1) \cdot (d + 1)$	$n + 1$
Block Ternary ($\ell = 4$)	$(1 + \frac{n}{4}) \cdot (d + 1)$	$(2n + 1) \cdot (d + 1)$	$2n + 1$
Block + Key unrolling	$(1 + \frac{n}{4}) \cdot (d + 1)$	$(2n + 1) \cdot (d + 1)$	$2n + 1$

5 Security, Parameters and Implementation

In this section, we give the security and experiments of our schemes. Our experiments compare the performance of our algorithms with NTRU-based GINX method [4] and the key unrolling method [27].

5.1 Security Analysis and Parameters

The security of our bootstrapping schemes mainly follows from the hardness of the LWE and NTRU problems, along with the *circular security assumption*. The latter is necessary because we encrypt the secret key f^{-1} using the key f itself during the initialization step.

The use of structured secret key distributions, such as block binary keys $B_{k,\ell}$ and block ternary keys $T_{k,\ell}$, reduces the size of the secret space compared to general binary keys. However, known attacks on LWE rely primarily on the standard deviation σ of the error distribution [1], rather than the entropy of the secret key, provided the key space remains sufficiently large.

A notable concern is the *meet-in-the-middle attack* [29], which recovers the secret key in time $O(S^{0.28})$, where S is the number of possible keys. To ensure 128-bit classical security, we impose the following constraints:

$$2^{0.28k \log(\ell+1)} \geq 2^{128} \text{ for } B_{k,\ell}, \text{ and } 2^{0.28k \log(2\ell+1)} \geq 2^{128} \text{ for } T_{k,\ell}.$$

There are some other possible attacks for block keys, see [25] for details.

Parameters We adopt LWE parameters based on the LWE standard [1] and NTRU parameters from [15]. The modulus Q , the dimension N , the LWE dimension n , and the gadget bases B , B_k are selected to ensure a minimum of 128-bit security. Due to the reduced key space in $B_{k,\ell}$ and $T_{k,\ell}$, we increase the LWE dimension n accordingly to offset entropy loss. Table 4 gives the detailed parameters.

Table 4. Parameters for LWE and NTRU ciphertexts

Method	Key distrib.	n	q	N	Q	B	Q_k	B_k
[4]	Binary	512	1024	1024	$2056193 \approx 2^{20.9}$	2^7	2^{14}	32
[27]	Binary	512	1024	1024	$2056193 \approx 2^{20.9}$	2^7	2^{14}	32
Block Binary ($\ell = 2$)	$\mathbf{B}_{n/2,2}$	576	1024	1024	$2056193 \approx 2^{20.9}$	2^7	2^{14}	32
Block Binary ($\ell = 3$)	$\mathbf{B}_{n/3,3}$	687	1024	1024	$2056193 \approx 2^{20.9}$	2^7	2^{14}	32
Block Binary ($\ell = 4$)	$\mathbf{B}_{n/4,4}$	788	1024	1024	$2056193 \approx 2^{20.9}$	2^7	2^{14}	32
Block Ternary ($\ell = 4$)	$\mathbf{T}_{n/4,4}$	576	1024	1024	$2056193 \approx 2^{20.9}$	2^7	2^{14}	32
Block + Key unrolling	$\mathbf{B}_{n/2,2}$	576	1024	1024	$2056193 \approx 2^{20.9}$	2^7	2^{14}	32

5.2 Implementation

We implement our bootstrapping algorithms based on the library CHIFHE [32] in C++ on a desktop with a 12th Gen Intel(R) Core(TM) i7-12700 @ 2.10 GHz CPU and 32 GB RAM, running Ubuntu 24.04. We compare the performance of previous algorithms to ours. All the experiments are done using a single thread. The times are running a single NAND gate. All the timing figures are the average times of 5000 experiments.

The key size includes both $\# \text{Memory} \cdot d \cdot N \cdot \log_2 Q$ bits bootstrapping keys and $(n+1) \cdot N \cdot B_k \cdot d_k \cdot \log_2 Q_k$ bits key-switch keys. In Table 5, one can see that the method of block ternary keys and key unrolling on block binary keys perform better than using block binary keys. The most important reason is they increase the size of bootstrapping keys instead of the dimension n . Since the size of key-switch keys are much larger than bootstrapping keys, our last 2 methods do not increase the key sizes too much but obviously improve the efficiency(1.12 times faster than key unrolling method [27] and 1.73 times faster than FINAL[4]).

Table 5. Times and Key sizes for different algorithms (one NAND gate)

Method	Key Sizes (MB)	Times (ms)
[4]	88.08	7.42
[27]	90.05	4.85
Block Binary ($\ell = 2$)	99.08	4.93
Block Binary ($\ell = 3$)	118.15	4.51
Block Binary ($\ell = 4$)	135.49	4.34
Block Ternary ($\ell = 4$)	103.48	4.29
Block + Key unrolling	103.48	4.26

6 Conclusion

In this paper, we proposed several techniques to enhance the efficiency of NTRU-based bootstrapping schemes, particularly targeting the blind rotation step, which constitutes the main computational bottleneck. For block binary keys, we slightly reduce the key space while maintaining the hardness of the LWE problem, and require no additional memory. For block ternary keys and combining key unrolling on block binary keys, we should increase some memory while keeping the security at the same level.

As in experimental results, using key unrolling on block binary keys is the most effective way (among our methods) to improve the efficiency of bootstrapping, without sacrificing security or drastically increasing resource requirements.

Acknowledgments. This work is supported by the Strategic Priority Research Program of the Chinese Academy of Sciences (Grant No. XDB0690200) and the National Natural Science Foundation of China (Grant No. 62372449).

References

1. Albrecht, M.R., Player, R., Scott, S.: On the concrete hardness of Learning with Errors. *Journal of Mathematical Cryptology* **9**(3), 169–203 (2015), <https://doi.org/10.1515/jmc-2015-0016>
2. Alperin-Sheriff, J., Peikert, C.: Faster Bootstrapping with Polynomial Error. In: Garay, J.A., Gennaro, R. (eds.) *Advances in Cryptology – CRYPTO 2014*. pp. 297–314. Springer Berlin Heidelberg, Berlin, Heidelberg (2014), https://doi.org/10.1007/978-3-662-44371-2_17
3. Bon, N., Pointcheval, D., Rivain, M.: Optimized Homomorphic Evaluation of Boolean Functions. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2024**(3), 302341 (Jul 2024), <https://tches.iacr.org/index.php/TCHES/article/view/11680>
4. Bonte, C., Iliashenko, I., Park, J., Pereira, H.V.L., Smart, N.P.: FINAL: Faster FHE Instantiated with NTRU and LWE. In: Agrawal, S., Lin, D. (eds.) *Advances in Cryptology – ASIACRYPT 2022*. pp. 188–215. Springer Nature Switzerland, Cham (2022), https://doi.org/10.1007/978-3-031-22966-4_7
5. Bourse, F., Minelli, M., Minihold, M., Paillier, P.: Fast Homomorphic Evaluation of Deep Discretized Neural Networks. In: Shacham, H., Boldyreva, A. (eds.) *Advances in Cryptology – CRYPTO 2018*. pp. 483–512. Springer International Publishing, Cham (2018), https://doi.org/10.1007/978-3-319-96878-0_17
6. Brakerski, Z.: Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In: Safavi-Naini, R., Canetti, R. (eds.) *Advances in Cryptology – CRYPTO 2012*. pp. 868–886. Springer Berlin Heidelberg, Berlin, Heidelberg (2012), https://doi.org/10.1007/978-3-642-32009-5_50
7. Brakerski, Z., Gentry, C., Vaikuntanathan, V.: (Leveled) fully homomorphic encryption without bootstrapping. In: *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*. p. 309325. ITCS '12, Association for Computing Machinery, New York, NY, USA (2012), <https://doi.org/10.1145/2090236.2090262>

8. Chen, B., Zhao, N.: Fully homomorphic encryption application in cloud computing. In: 2014 11th International Computer Conference on Wavelet Active Media Technology and Information Processing (ICCWAMTIP). pp. 471–474 (2014). <https://doi.org/10.1109/ICCWAMTIP.2014.7073452>
9. Cheon, J.H., Han, K., Kim, A., Kim, M., Song, Y.: Bootstrapping for Approximate Homomorphic Encryption. In: Nielsen, J.B., Rijmen, V. (eds.) *Advances in Cryptology – EUROCRYPT 2018*. pp. 360–384. Springer International Publishing, Cham (2018), https://doi.org/10.1007/978-3-319-78381-9_14
10. Cheon, J.H., Kim, A., Kim, M., Song, Y.: Homomorphic Encryption for Arithmetic of Approximate Numbers. In: Takagi, T., Peyrin, T. (eds.) *Advances in Cryptology – ASIACRYPT 2017*. pp. 409–437. Springer International Publishing, Cham (2017), https://doi.org/10.1007/978-3-319-70694-8_15
11. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster Fully Homomorphic Encryption: Bootstrapping in Less Than 0.1 Seconds. In: Cheon, J.H., Takagi, T. (eds.) *Advances in Cryptology – ASIACRYPT 2016*. pp. 3–33. Springer Berlin Heidelberg, Berlin, Heidelberg (2016), https://doi.org/10.1007/978-3-662-53887-6_1
12. Chillotti, I., Gama, N., Georgieva, M., Izabachène, M.: Faster Packed Homomorphic Operations and Efficient Circuit Bootstrapping for TFHE. In: Takagi, T., Peyrin, T. (eds.) *Advances in Cryptology – ASIACRYPT 2017*. pp. 377–408. Springer International Publishing, Cham (2017), https://doi.org/10.1007/978-3-319-70694-8_14
13. Corrigan-Gibbs, H., Henzinger, A., Tauman Kalai, Y., Vaikuntanathan, V.: Somewhat Homomorphic Encryption from a Linear Homomorphism and a Sparse LPN. In: Fehr, S., Fouque, P.A. (eds.) *Advances in Cryptology – EUROCRYPT 2025*. pp. 3–33. Springer Nature Switzerland, Cham (2025), https://doi.org/10.1007/978-3-031-91124-8_1
14. Ducas, L., Micciancio, D.: FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In: Oswald, E., Fischlin, M. (eds.) *Advances in Cryptology – EUROCRYPT 2015*. pp. 617–640. Springer Berlin Heidelberg, Berlin, Heidelberg (2015), https://doi.org/10.1007/978-3-662-46800-5_24
15. Ducas, L., van Woerden, W.: NTRU Fatigue: How Stretched is Overstretched? In: Tibouchi, M., Wang, H. (eds.) *Advances in Cryptology – ASIACRYPT 2021*. pp. 3–32. Springer International Publishing, Cham (2021), https://doi.org/10.1007/978-3-030-92068-5_1
16. Fan, J., Vercauteren, F.: Somewhat Practical Fully Homomorphic Encryption. *Cryptology ePrint Archive*, Paper 2012/144 (2012), <https://eprint.iacr.org/2012/144>
17. Gama, N., Izabachène, M., Nguyen, P.Q., Xie, X.: Structural Lattice Reduction: Generalized Worst-Case to Average-Case Reductions and Homomorphic Cryptosystems. In: Fischlin, M., Coron, J.S. (eds.) *Advances in Cryptology – EUROCRYPT 2016*. pp. 528–558. Springer Berlin Heidelberg, Berlin, Heidelberg (2016), https://doi.org/10.1007/978-3-662-49896-5_19
18. Gentry, C.: Fully homomorphic encryption using ideal lattices. In: *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*. p. 169178. STOC '09, Association for Computing Machinery, New York, NY, USA (2009), <https://doi.org/10.1145/1536414.1536440>
19. Gentry, C., Sahai, A., Waters, B.: Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based. In: Canetti, R., Garay, J.A. (eds.) *Advances in Cryptology – CRYPTO 2013*. pp. 75–92. Springer Berlin Heidelberg, Berlin, Heidelberg (2013), https://doi.org/10.1007/978-3-642-40041-4_5

20. Halevi, S., Shoup, V.: Bootstrapping for HELib. *J. Cryptol.* **34**(1) (Jan 2021). <https://doi.org/10.1007/s00145-020-09368-7>
21. He, H., Wang, Z., Jain, H., Jiang, C., Yang, S.: A privacy-preserving decentralized credit scoring method based on multi-party information. *Decision Support Systems* **166**, 113910 (2023), <https://www.sciencedirect.com/science/article/pii/S0167923622001816>
22. Hoffstein, J., Pipher, J., Silverman, J.H.: NTRU: A ring-based public key cryptosystem. In: Buhler, J.P. (ed.) *Algorithmic Number Theory*. pp. 267–288. Springer Berlin Heidelberg, Berlin, Heidelberg (1998), <https://doi.org/10.1007/BFb0054868>
23. Jadoul, R., Mertens, A., Park, J., Pereira, H.V.L.: NTRU-Based FHE for Larger Key and Message Space. In: Zhu, T., Li, Y. (eds.) *Information Security and Privacy*. pp. 141–160. Springer Nature Singapore, Singapore (2024), https://doi.org/10.1007/978-981-97-5025-2_8
24. Joye, M., Paillier, P.: Blind Rotation in Fully Homomorphic Encryption with Extended Keys. In: Dolev, S., Katz, J., Meisels, A. (eds.) *Cyber Security, Cryptology, and Machine Learning*. pp. 1–18. Springer International Publishing, Cham (2022), https://doi.org/10.1007/978-3-031-07689-3_1
25. Lee, C., Min, S., Seo, J., Song, Y.: Faster TFHE Bootstrapping with Block Binary Keys. In: *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*. pp. 2–13. ASIA CCS '23, Association for Computing Machinery, New York, NY, USA (2023), <https://doi.org/10.1145/3579856.3595804>
26. Lee, Y., Micciancio, D., Kim, A., Choi, R., Deryabin, M., Eom, J., Yoo, D.: Efficient FHEW Bootstrapping with Small Evaluation Keys, and Applications to Threshold Homomorphic Encryption. In: Hazay, C., Stam, M. (eds.) *Advances in Cryptology – EUROCRYPT 2023*. pp. 227–256. Springer Nature Switzerland, Cham (2023), https://doi.org/10.1007/978-3-031-30620-4_8
27. Li, Z., Lu, X., Wang, Z., Wang, R., Liu, Y., Zheng, Y., Zhao, L., Wang, K., Hou, R.: Faster NTRU-based Bootstrapping in less than 4 ms. *IACR Transactions on Cryptographic Hardware and Embedded Systems* **2024**(3), 418451 (Jul 2024), <https://tches.iacr.org/index.php/TCHES/article/view/11683>
28. Lyubashevsky, V., Peikert, C., Regev, O.: On Ideal Lattices and Learning with Errors over Rings. In: Gilbert, H. (ed.) *Advances in Cryptology – EUROCRYPT 2010*. pp. 1–23. Springer Berlin Heidelberg, Berlin, Heidelberg (2010), https://doi.org/10.1007/978-3-642-13190-5_1
29. May, A.: How to Meet Ternary LWE Keys. In: Malkin, T., Peikert, C. (eds.) *Advances in Cryptology – CRYPTO 2021*. pp. 701–731. Springer International Publishing, Cham (2021), https://doi.org/10.1007/978-3-030-84245-1_24
30. Regev, O.: On lattices, learning with errors, random linear codes, and cryptography. *J. ACM* **56**(6) (Sep 2009), <https://doi.org/10.1145/1568318.1568324>
31. Wang, R., Wen, Y., Li, Z., Lu, X., Wei, B., Liu, K., Wang, K.: Circuit Bootstrapping: Faster and Smaller. In: Joye, M., Leander, G. (eds.) *Advances in Cryptology – EUROCRYPT 2024*. pp. 342–372. Springer Nature Switzerland, Cham (2024), https://doi.org/10.1007/978-3-031-58723-8_12
32. Xiang, B.: CHIFHE (2023), <https://github.com/SKLC-FHE/CHIFHE>, last accessed on May 17, 2025
33. Xiang, B., Zhang, J., Deng, Y., Dai, Y., Feng, D.: Fast Blind Rotation for Bootstrapping FHEs. In: Handschuh, H., Lysyanskaya, A. (eds.) *Advances in Cryptology – CRYPTO 2023*. pp. 3–36. Springer Nature Switzerland, Cham (2023), https://doi.org/10.1007/978-3-031-38551-3_1

A Previous Bootstrapping Algorithms

We give some previous algorithms which occur in our Implementation section.

Algorithm 4 Original NTRU-based Bootstrapping (GINX method)[4]

Input: An LWE sample $\text{ct} = (\mathbf{a}, b = -\langle \mathbf{a}, \mathbf{s} \rangle - \lfloor \frac{q}{t} \rfloor \cdot m + e) \in \text{LWE}_{\mathbf{s},q}^n(m)$.

The bootstrapping key $\{\mathbf{evk}_i\}, 0 \leq i \leq n$.

A special bootstrapping key $\mathbf{evk}' = \text{VNTRU}_{f,Q}(f^{-1})$.

An LWE key switching key $\mathbf{ksk}_{\mathbf{s},(\phi(h))}$.

Output: An LWE sample $\text{ct}' \in \text{LWE}_{\mathbf{s},q}^n(h(m))$.

1: Set $P(X) = \sum_{i=0}^{N-1} \frac{Q}{t} \cdot h\left(\left\lfloor \frac{t}{q} \cdot i \right\rfloor\right) \cdot X^i$, and $Y = X^{\frac{2N}{q}}$

2: Let $\text{ACC} = (P(X) \cdot Y^b) \odot \mathbf{evk}'$

for $i = 0; i < n; i = i + 1$ **do**

$$\text{ACC} = \text{ACC} + \text{ACC} \odot (Y^{a_i} - 1) \cdot \mathbf{evk}_i$$

end for

3: $\text{ct}' \leftarrow \text{Ext}(\text{ACC})$

4: $\text{ct}' \leftarrow \text{ModSwitch}_{Q \rightarrow Q_k}(\text{ct}')$

5: $\text{ct}' \leftarrow \text{LWE.KeySwitch}(\text{ct}')$

6: $\text{ct}' \leftarrow \text{ModSwitch}_{Q_k \rightarrow q}(\text{ct}')$

return ct'

Algorithm 5 NTRU-based Bootstrapping using key unrolling [27]

Input: An LWE sample $\text{ct} = (\mathbf{a}, b = -\langle \mathbf{a}, \mathbf{s} \rangle - \lfloor \frac{q}{t} \rfloor \cdot m + e) \in \text{LWE}_{\mathbf{s},q}^n(m)$.

The bootstrapping key $\{\mathbf{evk}_{i,j}\}, 0 \leq i \leq n/2, 0 \leq j \leq 2$.

A special bootstrapping key $\mathbf{evk}' = \text{VNTRU}_{f,Q}(f^{-1})$.

An LWE key switching key $\mathbf{ksk}_{\mathbf{s},(\phi(h))}$.

Output: An LWE sample $\text{ct}' \in \text{LWE}_{\mathbf{s},q}^n(h(m))$.

1: Set $P(X) = \sum_{i=0}^{N-1} \frac{Q}{t} \cdot h\left(\left\lfloor \frac{t}{q} \cdot i \right\rfloor\right) \cdot X^i$, and $Y = X^{\frac{2N}{q}}$

2: Let $\text{ACC} = (P(X) \cdot Y^b) \odot \mathbf{evk}'$

for $i = 0; i < n/2; i = i + 1$ **do**

$$\begin{aligned} \text{ACC} = \text{ACC} + \text{ACC} \odot [(Y^{a_{2i}+a_{2i+1}} - 1) \cdot \mathbf{evk}_{i,0} \\ + (Y^{a_{2i}} - 1) \cdot \mathbf{evk}_{i,1} + (Y^{a_{2i+1}} - 1) \cdot \mathbf{evk}_{i,2}]. \end{aligned}$$

end for

3: $\text{ct}' \leftarrow \text{Ext}(\text{ACC})$

4: $\text{ct}' \leftarrow \text{ModSwitch}_{Q \rightarrow Q_k}(\text{ct}')$

5: $\text{ct}' \leftarrow \text{LWE.KeySwitch}(\text{ct}')$

6: $\text{ct}' \leftarrow \text{ModSwitch}_{Q_k \rightarrow q}(\text{ct}')$

return ct'
